

Föreläsning 17

Ändrad
3 Oct
16:05

Återigen ett exempel som visar släktskapet mellan pekare och fält. Programmet skriver ut `x[5]`, `x[7]` och `x[1]`. Varje gång på fyra olika sätt.

```

1  #include "stdio.h"
2
3  void main()
4  {
5      double *p, x[10] = {0, 10, 20, 30, 40,
6                          50, 60, 70, 80, 90};
7      p = &x[5];
8
9      printf("%g %g %g %g\n", x[5], *(x+5),      *p, p[0]);
10
11     printf("%g %g %g %g\n", x[7], *(x+7), *(p+2), p[2]);
12
13     printf("%g %g %g %g\n", x[1], *(x+1), *(p-4), p[-4]);
14 }

```

Programmet fungerar:

```

1  50 50 50 50
2  70 70 70 70
3  10 10 10 10

```

Repetition pekare och fält:

- `x[0]`, `x[1]`, etc, lagras konsekutivt
- `x` är en pekare på `double`. Precis som `p`
- `x` är alltid lika med `&x[0]`
- `&x[3]` står för adressen till `x[3]`
- Pekararitmetik översätter `p+3` till `p+3*sizeof(double)`
- `p[3]` är exakt detsamma som `*(p+3)`
- `*p` är den `double` som ligger på adress `p` och några celler fram.

Strängar

En sträng (en *string*) är en sekvens tecken. Till exempel ett namn, eller en rad med text. Vi har tidigare sett strängar i anropen till `scanf()` och `printf()`, mellan citationstecken: `"abc"`.

Egentligen är en sträng bara ett `char`-fält. Men programspråket C är extrautrustat för att hantera just strängar.

Det konventionella korta variabelnamnet på en sträng är `s` (dvs första bokstaven i "string"):

```
char s[200];
```

Funktionerna `printf()` och `scanf()` kan känner till strängar. Dessutom finns funktionerna `gets()` och `puts()`

(analogt med `getchar()` och `putchar()`).

Det ska inte vara `&` i anropen till `scanf` och `gets()` eftersom `s` är en pekare:

```
1 #include "stdio.h"
2
3 void main()
4 {
5     char s[100];
6
7     gets(s);
8     printf("%s\n", s);
9
10    scanf("%s", s);
11    puts(s);
12 }
```

Utskriften visar att `scanf()` och `gets()` inte fungerar likadant:

```
1 En bro en sko
2 En bro en sko
3 en gren en sten
4 en
```

Som synes läser `gets()` in en hel rad medan `scanf()` bara läser in fram till nästa tomrum (eng *white space*, dvs blank, ny rad, eller tabulator).

Enda skillnaden mellan `printf()` och `puts()` är att `puts()` automatiskt lägger till en radframmatning.

Men -- förresten -- hur kan `printf()` och `puts()` veta att de inte ska skriva ut alla 100 tecken i `s`? Jo -- därför att `scanf()` och `gets()` sätter dit ett extra tecken i slutet på strängen: en nolla som markerar strängslut.

'\0' = 0 markerar strängens slut

Det är en konvention att alla strängar avslutas med `char`-värdet `0 = '\0'`. (Obs inte `'0'`, utan heltalet `0` -- tecknet med ASCII-värdet noll).

Funktionerna `scanf()` och `gets()` följer denna konvention. När de läst färdigt användarens inmatning lägger de till ett `'\0'` på slutet. Och `printf()` och `puts()` vet att de ska sluta skriva ut när `'\0'` dyker upp.

```
1 #include "stdio.h"
2
3 void main()
4 {
5     char s[10] = "aaabbbccc";
6     int i;
7
8     puts(s);
9     gets(s);
10    puts(s);
11
12    for(i = 0; i < 10; i++)
13        printf("s[%d] = %d (%c)\n", i, s[i], s[i]);
14    printf("\n");
15 }
```

■ Lagg märke till det korta sättet att initiera strängar. Programmeraren använder -- fullt korrekt -- en sträng med nio tecken, men bokar plats för tio -- en extra för den

avslutande nollan.

■ I utskriften ser vi att att `gets()` har skrivit över initieringens värden, och lagt till en nolla på slutet. Resten av initieringen ligger kvar orörd, inklusive en avslutande nolla:

```

1  aaabbbcccc
2  HEJ!
3  HEJ!
4  s[0] = 72 (H)
5  s[1] = 69 (E)
6  s[2] = 74 (J)
7  s[3] = 33 (!)
8  s[4] = 0  ()
9  s[5] = 98 (b)
10 s[6] = 99 (c)
11 s[7] = 99 (c)
12 s[8] = 99 (c)
13 s[9] = 0  ()

```

Egen `gets()`

Det står oss många möjligheter till buds om vi vill läsa in text. Vi kan läsa in tecken för tecken med `getchar()`, ord för ord med `scanf()`, eller rad för rad med `gets()`.

Jag har implementerat tre egna versioner av `gets()`. Det är bara den tredje versionen som fungerar som originalet.

```

1  #include <stdio.h>
2
3  void gets1(char s[])
4  {
5      int i = 0;
6      while ((s[i] = getchar()) != '\n')
7          i++;
8      s[i] = 0;
9  }
10
11 void gets2(char *s)
12 {
13     while ((*s = getchar()) != '\n')
14         s++;
15     *s = 0;
16 }
17
18 char *gets3(char *s)
19 {
20     char *p = s;
21     while ((*s = getchar()) != '\n')
22         s++;
23     *s = 0;
24     return p;
25 }
26
27 void main()
28 {
29     char s[1000];
30     gets1(s);
31     puts(s);
32     gets2(s);
33     puts(s);
34     puts(gets3(s));
35 }

```

■ Funktionen `gets1()` läser in tecken till `s` till dess `'\n'` dyker upp. På rad 8 ersätts det sist inlästa tecknet `'\n'` med `0`.

■ Funktionen `gets2()` tar emot en pekare på `char` som

argument -- pekare och fält är samma sak.

Vi använder `*`-operatoren för att komma åt de enskilda `char`-variablerna i strängen (rad 6). Och vi använder pekarstegning för att ta oss fram till nästa tecken i strängen (rad 7).

■ `gets1()` och `gets2()` skiljer sig från originalet i det att de inte returnerar ett värde. De borde (enligt originalet) returnera ett värde av typen `char`-pekare.

Det gör `gets3()`. Returvärde är argumentet till funktionen. Jag lagrar argumentet i den lokala variabeln `p`. Och avslutar med att returnera `p`. Anropet till `gets3()` utnyttjar returvärdet som parameter till `puts()`.

■ Inte ens den tredje versionen fungerar *exakt* som originalet -- vi återkommer till det.

```
1 Jag får en knuff av en riktig man
1 Jag får en knuff av en riktig man
2 Han brukar tränga sig före ibland
2 Han brukar tränga sig före ibland
3 det gör mig ingenting
3 det gör mig ingenting
```

! Det är upp till programmeraren att se till att det finns tillräckligt med plats i strängen `s`.
! Både min `gets()` och originalet läser in tecken tills `'\n'` dyker upp. Oavsett om cellerna är bokade eller ej.

Egen `puts()`:

`puts()` håller utkik efter `'\0'`:

```
1 #include <stdio.h>
2
3 char *puts1(char s[])
4 {
5     char *p = s;
6     int i = 0;
7     while (s[i] != '\0')
8         putchar(s[i++]);
9     putchar('\n');
10    return p;
11 }
12
13 char *puts2(char *s)
14 {
15     char *p = s;
16     while (*s != '\0')
17         putchar(*(s++));
18     putchar('\n');
19     return p;
20 }
21
22 void main()
23 {
24     char s[1000];
25
26     puts1(gets(s));
27     puts2(gets(s));
28 }
```

Input/output:

```
1 Under tapeten en gammal tapet
```

```
2 Under tapeten en gammal tapet
3 det gör mig ingenting.
4 det gör mig ingenting.
```

Nästa lektion

